National University of Computer and Emerging Sciences, Lahore Campus

	Course:	Parallel and Distributed Computing	Course Code:	CS3006
	Program:	BS(CS)-7G	Semester:	Fall 2026
	Due Date:	Wednesday, 14 October 2026	Total Marks:	100 marks
	Instructor : Type:	Mubashar Hussain Assignment 2	TA:	l232597@lhr.nu.edu.pk

Building a Parallel Task Execution Library

This is an individual assignment. Do not collaborate on code or write-ups.

Estimated Time to Complete: 10–12 hours (spread across the week)

1. Objectives

- Build a multi-core task execution library and progressively improve it from repeated thread creation to persistent worker-thread pools.
- Understand how task granularity, workload imbalance, scheduling overhead, and synchronization affect parallel performance.
- Use mutexes and condition variables to coordinate worker threads correctly and avoid wasting CPU resources while idle.
- Measure runtime, speedup, and efficiency and use experimental evidence to explain why scaling improves, saturates, or degrades.
- Extend the runtime to support asynchronous bulk task launches with explicit dependencies between launches.
- Analyze dependency graphs in terms of ready work, blocked work, critical path, available concurrency, and scheduling constraints.

2. Prerequisites & Setup

- A C++ toolchain with C++11 thread support on Linux/macOS, or WSL on Windows. •

Basic Python/matplotlib, gnuplot, or Excel for plotting performance graphs.

- You have been provided a starter-code archive. Keep the supplied folder structure, the interfaces in itasksys.h, and the Makefiles unchanged. Do not modify or remove the supplied tests.
- Your implementation goes in tasksys.cpp and tasksys.h of the relevant folder. You may add member variables, helper classes, and helper functions there. Do not change public method signatures.

• Your own tests are added to tests/tests.h (see the yourTest() skeleton) and registered in tests/main.cpp. This is the only change allowed outside the task-system source files.

The starter code provides IRunnable and ITaskSystem interfaces, a correct serial task system, three incomplete parallel task-system classes, a test suite, and an asynchronous dependency interface. You will improve the runtime in stages so that every stage remains a correct task execution system. Parts 1–4 are implemented in part1to4 synchronous/. Part 5 is imple mented in part5 async deps/, whose sleeping thread-pool class must contain your complete implementation (including a correct run()) plus the asynchronous methods.

3. The Tasks

Part 1: Synchronous Parallel Task Launch (20 marks)

A bulk launch consists of many instances of the same logical task. The serial reference executes every task on the calling thread. Your first goal is to execute a bulk launch in parallel while preserving synchronous semantics: when run() returns, every task in that launch must have finished.

Implement: TaskSystemParallelSpawn::run()

Work through these stages in order

- Create worker threads for a call to run() and join them before returning. The number of threads you create must never exceed the num threads value passed to the constructor (the calling thread is not counted; it may simply wait for the workers).
- Design a correct mechanism for assigning task IDs to workers. Your implementation must execute every task exactly once.
- Identify all shared scheduler state. Protect state that can be accessed concurrently.
- Test correctness on workloads with very few tasks, many tasks, and task counts that are not divisible by the number of workers.
- Create one additional test that stresses either correctness or performance.

Performance investigation
Choose at least three supplied workloads: one with very small tasks, one with substantial work per task, and one with non-uniform task cost. For each workload run with 1, 2, 4, 8, and (if supported by your machine) 16 workers. Record:

- Before running the experiments, predict which workload will scale best and which will scale worst. Give a technical reason.
- Identify the first worker count for which adding workers improves runtime by less than 10%. Explain the cause using your measurements.
- For the irregular workload, explain whether static or dynamic assignment would be expected to balance work better. Your answer must discuss both load balance and scheduling cost.
- Give one measured example in which parallel execution is correct but provides little or no speedup. Explain why.

Deliverable
Completed implementation, your added test, a timing/speedup/efficiency table, and a concise

analysis supported by your own measurements.

Part 2: Persistent Thread Pool – Spinning (20 marks)

Creating and destroying worker threads for every bulk launch can be expensive, particularly when individual tasks are short. Replace repeated thread creation with a persistent pool of workers.

Implement: TaskSystemParallelThreadPoolSpinning

- Create the worker threads once and keep them alive across multiple calls to run().
- Workers may spin while waiting for new work in this part.
- run() must still be synchronous. It must not return until the current bulk launch is com pletely finished.
- Your task system must remain correct for consecutive calls to run(), including launches with different task counts.
- Ensure destruction of the task system terminates all workers cleanly.

Analysis

- Compare ParallelSpawn and the spinning thread pool on at least one fine-grained and one coarse-grained workload. Explain why eliminating thread creation changes the two workloads differently.
- Measure a case with little or no work available for some workers. Explain what the idle spinning workers are doing and why this is a resource cost even if correctness is unaffected.
- Suppose the spinning pool is faster than the spawn implementation but efficiency still de creases as p increases. Explain how both observations can be true at the same time.
- Using your measurements, separate thread-management overhead from useful task compu tation as far as your data allows. State any limitation in your conclusion.

Deliverable
Completed spinning thread pool and a comparison against Part 1 using measured evidence.

Part 3: Sleeping Thread Pool and Synchronization (20 marks)

A spinning worker consumes execution resources even when no useful task is available. Improve the thread pool so workers sleep while waiting and are awakened when the state they are waiting for changes.

Implement: TaskSystemParallelThreadPoolSleeping

- Use appropriate synchronization primitives such as mutexes and condition variables. •

Workers should sleep when there is no runnable work rather than continuously polling. •

The calling thread should wait efficiently for completion of a synchronous bulk launch. •

Handle wakeups and shared state correctly under arbitrary thread interleavings.

• Do not replace one busy-wait loop with another busy-wait loop hidden behind a different condition.

Synchronization reasoning

• Describe the condition on which a worker sleeps and the event that makes that condition potentially true.

• Describe the condition on which the calling thread waits for run() to complete.

• Give one incorrect interleaving that could cause run() to return before all tasks finish if completion state is updated incorrectly.

• Explain why checking a condition in a loop is necessary when using condition variables.

• Compare spinning and sleeping on both a fine-grained workload and a workload containing idle gaps. Explain why the same implementation need not be fastest in both cases.

Deliverable
Completed sleeping thread pool, synchronization explanation, and measured comparison of Spawn vs. Spin vs. Sleep.

Part 4: Scheduling and Granularity Investigation (15 marks)

In this part you will use the task system you have already built; you are not writing a separate scheduler. Add a test workload in which task costs are intentionally non-uniform. The goal is to demonstrate how mapping and granularity affect makespan and worker utilization.

• With four workers, draw a possible timeline for a static contiguous assignment. Calculate the makespan and utilization of each worker.

• Draw a possible timeline for a dynamic next-task assignment. Calculate its makespan and compare it with the static case.

• Now combine several small tasks into larger chunks. Experiment with at least three chunk sizes.

• Identify a chunk size that reduces scheduling overhead and another that reduces available concurrency or worsens load balance. Explain why.

• Let H denote average scheduling/coordination overhead per scheduled unit and T denote average useful computation per unit. Use the ratio H/T to explain why very fine granularity can destroy the benefit of dynamic scheduling.

Example pattern (you may use an equivalent irregular pattern in which static and dynamic assignment give different makespans):

Task ID: 0 1 2 3 4 5 6 7

Cost (ms): 1 1 1 1 10 10 10 10

Deliverable
The added irregular workload, execution timelines, utilization/makespan calculations, and a granularity experiment.

Part 5: Asynchronous Task Graphs with Dependencies (25 marks)

The final part extends your sleeping thread-pool runtime beyond one synchronous bulk launch at a time. Applications may submit several bulk launches asynchronously, and a launch may depend on the completion of earlier launches.

Implement: runAsyncWithDeps() and sync() in the sleeping thread-pool task system (in part5 async deps/).

- runAsyncWithDeps() must return without waiting for the submitted launch to complete. •

Each submitted bulk launch must receive a unique TaskID.

- A launch with no unsatisfied dependencies may become runnable immediately.
- A launch with dependencies must not execute any of its tasks until every launch listed in its dependency set has completed.
- sync() must return only after all previously submitted asynchronous launches have com pleted.
- Your runtime should allow independent ready launches to make progress concurrently when worker capacity is available.
- run() in this class must remain a correct synchronous launch.

Dependency analysis

- State which launches can be ready at the same time and which ordering constraints are mandatory.
- Assume the average cost of one task in A, B, C and D is 1 ms, 8 ms, 4 ms and 2 ms respectively. Calculate the total work represented by each launch.
- Using the dependency graph and the supplied costs, determine a critical-path lower bound. Clearly state the assumptions used in your calculation.
- Construct a valid execution schedule for 2 workers and another for 4 workers. Your schedule must respect both dependencies and worker capacity.
- Explain, using this graph, why doubling the number of workers does not necessarily halve completion time.
- Describe what bookkeeping must occur when the final task of a bulk launch completes. Which blocked launches can change state at that instant?
- Give one race condition that could allow a dependent launch to start early, and state the invariant your implementation uses to prevent it.
- Split one long launch into four independent sub-launches and discuss whether the additional concurrency is always beneficial once launch/scheduling overhead is included.

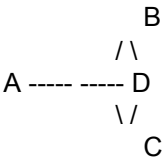
Dependency graph for the analysis above:

A: 128 tasks, no dependencies

B: 2 tasks, depends on A

C: 6 tasks, depends on A

D: 32 tasks, depends on B and C



Deliverable

Completed asynchronous dependency implementation, one custom dependency test, DAG cal culations/schedules, and an explanation of the runtime's dependency bookkeeping.

4. Combined Performance Analysis

Submit one combined PDF report. The purpose of the report is to explain the behavior of your own implementation, not to repeat definitions from lecture slides.

- Machine description: CPU model, number of physical cores and hardware threads, oper ating system, and compiler version. Worker counts above your core count must be interpreted with this in mind.
- Runtime vs. number of workers for ParallelSpawn, ThreadPoolSpinning, and ThreadPool Sleeping.
- Speedup vs. number of workers for the same implementations.
- Efficiency vs. number of workers for at least three workloads with different characteristics.
- A table containing T_1, T_p, S_p and E_p for all reported measurements.
- Measurement method: for every reported time, take the minimum or the median of at least 5 runs, state which statistic you used, and use the same one throughout. Define T_1 explicitly: either the runtime of the supplied serial implementation, or the runtime of the same implementation with $p = 1$. Use one definition consistently and state it.
- One prediction you made before experimentation that was confirmed, and one prediction that was contradicted by the measurements. Explain the latter. If none of your predictions was contradicted, say so and explain which prediction you were least sure of and why.

- For each workload, identify the dominant limitation from: insufficient concurrency, load im balance, task granularity, thread-management overhead, synchronization/queue contention, or idle waiting. Support the diagnosis with data.
- Conclude by describing workload conditions under which each of the three task-system de signs is appropriate. There is no single universally best implementation.
- Correctness evidence: include the output (text or screenshot) of the supplied tests passing for each implementation, either in the report or as an appendix.

Speedup: $S_p = T_1/T_p$ Efficiency: $E_p = S_p/p$

5. Deliverables Checklist

- Completed source code for all required task-system implementations.
- At least one custom synchronous test and one custom dependency test, added to tests/tests.h and registered in tests/main.cpp.
- Correctness evidence from the supplied test suite.
- One combined PDF report containing all required tables, graphs, timelines, DAG analysis, calculations, predictions, and explanations.
- README containing only the commands required to build and run your solution (replace the README supplied in the starter code).
- All files zipped as RollNumber Assignment2.zip. The zip must contain, in its top-level folder: the starter-code folder structure (part1to4 synchronous/, part5 async deps/, common/, tests/), your report as RollNumber Report.pdf, and your README. Delete objs/ and compiled binaries before zipping.

6. Tips for Success

- Get one implementation fully correct before moving to the next. The stages intentionally build on one another.
- Use small correctness tests before performance tests. Timing a race condition only gives you a fast incorrect program.
- Run timing experiments multiple times and use a consistent statistic (median or minimum, see Section 4) because parallel timings contain noise.
- Draw the state transitions of a worker and the dependency graph on paper before imple menting complex synchronization.
- Record predictions before running an experiment. The difference between prediction and observation is often where the most useful performance insight appears.
- Do not report ‘overhead’ as a complete explanation. Identify which overhead: thread cre ation, scheduling, synchronization, queue contention, wakeup cost, or insufficient useful work per task.